program when you run it. This is accomplished by passing command line arguments to **main()**. A command line argument is the information which directly follows the name of the program on the command line, when the program is executed. In Java, the command line arguments are stored as strings in the **String** array passed to **main()**. Therefore they can be easily accessed as shown in the following example :

```
class CommandDemo {
    public static void main(String args[ ]) {
        for(int i = 0; i < args.length; i++)
            System.out.println("args[" + i + "]" : " +
args[i]);
    }
}
```

If you run this program at the command line as follows :

java CommandDemo Passing Command line arguments 1

then the output will be :

args[0] : Passing
args[1] : Command
args[2] : line
args[3] : arguments
args[4] : 1

Remember, command line arguments are passed as strings, so you should convert numeric values to their internal forms manually.

8.6 to 8.8 Check Your Progress.

1. Fill in the blanks.

a) ..... searches for the first occurance of a character or a substring.

b) ..... method returns a copy of the invoking string from which any leading and trailing white spaces have been removed.

c) ..... converts all the characters in a string from lowercase to uppercase.

d) Command line arguments are passed as .....

8.9 SUMMARY

In Java a string is a sequence of characters. Java implements strings as objects of type String. String objects an be constructed in a number of ways, therefore it is easy to obtain a string when needed. Java has a number of methods to perform various operations on strings like comparing strings, concatenating strings, changing case of letters within a string etc.

Source : *train-srv.manipalu.com (Link)*

8.10 CHECK YOUR PROGRESS - ANSWERS

8.1

1. a) String

b) String

c) StringBuffer

8.2

1. a) String()

b) String(char chars[])

c) String(byte asciiChars[])

8.3

1. a) False

b) True

c) True

8.4

1. a) iii)

b) i)

c) ii)

8.5

1. a) True

b) True

c) True

8.6 to 8.8

1. a) indexOf()

b) trim()

c) toUpperCase()

d) strings

8.11 QUESTIONS FOR SELF - STUDY

1. Compare String and StringBuffer class.
2. Describe the various constructors of String.

3. Write short notes on :

   a) String Literals

   b) toString()

   c) Command Line arguments.

4. Describe the various ways in which characters can be extracted from a String object.

5. Describe the various functions which are used to compare Strings in Java.

6. Describe the methods useful for searching Strings.

7. Which methods are useful for modifying Strings? Explain them.

8.12 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net

❑ ❑ ❑

Chapter : 9

Input/Output

9.0 OBJECTIVES

Dear Students,

After studying this chapter you will able to :

- discuss an overview of user interaction in Java
- discribe what are streams
- discuss methods for performing console input and console output
- discuss file input and output

9.1 INTRODUCTION

In Java, user interaction takes place through graphically oriented **applets** which depend upon the Abstract Window Toolkit. Thus most real applications of Java are not text based console programs. Also Java provides limited support to console Input/Output and it is not very important to Java programming. However, Java provides a strong flexible support for I/O as it relates to files and networks. In this chapter, let us familiarize ourselves with console and file input/output in Java.

9.2 STREAMS

I/O is performed through streams in Java. A stream is an abstraction which either produces or consumes information. A stream is linked to a physical device by the Java I/O system. The same I/O classes and methods can be applied to any

type of device, because all the streams behave in the same manner irrespective of the physical devices to which they connect. An input stream can thus abstract various kinds of input like disk file, keyboard, network socket etc. Similarly an output stream may refer to the console, a disk file or a network connection. Java implements streams within class hierarchies defined in the **java.io** package.

Java defines two types of streams : **byte streams** and **character streams**. Byte streams provide a convenient way to handle input/output of bytes. They are useful when reading or writing binary data. Character streams provide a convenient way of handling input and output of characters. They use Unicode and can therefore be internationalised. In some cases, character streams are more efficient than byte streams. Note however, that at the lowest level all I/O is byte oriented. The character based I/O streams are just a convenient and efficient way to handle characters.

9.2.1 The Byte Stream Classes

Byte streams are defined by using two class hierarchies. At the top are two abstract classes : **InputStream** and **OutputStream**. Both these classes have a number of concrete subclasses which handle the differences between various devices like disks, network connections and memory buffers.

Some of the byte stream classes which we shall use in our examples are :

BufferedInputStream	Buffered Input stream
BufferedOutputStream	Buffered Output stream
DataInputStream	Input stream that contains methods for reading the Java standard data types
DataOutputStream	Output stream that contains mehtods for writing the Java standard data types

In order to use the stream classes you have to import **java.io**. There are abstract classes in **InputStream** and **OutputStream** which define several key methods which the other stream classes implement. The most important methods are **read()** which is used to read bytes of data and **write()** which is used to write bytes of data. Both the methods are declared **abstract**. They are overridden by the derived stream classes.

9.2.2 Character Stream Classes

They are also defined by using two class hierarchies. At the top are two abstract classes **Reader** and **Writer**. These classes handle the Unicode character streams. There are a number of concrete subclasses of each of these. Some of the character stream classes are :

The **Reader** and **Writer** classes define several methods which are implemented by the other stream classes. The most common among them are **read()** and **write()** which read and write characters of data, respectively.These methods are overridden by the derived stream classes.

Buffered Reader	Buffered Input Character Stream
Buffered Writer	Buffered output Character Stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file

9.2.3 Predefined Streams

We know that all Java programs automatically import the **java.lang** package. This package defines a class called **System**. **System** encapsulates several aspects of the Java run time environment. eg. with some of its method you can obtain the current time, the settings of various properties associated with the system etc. **System** contains three predefined stream variables **in**, **out** and **err** which are declared **public** and **static** within **System**. Thus they can be used by any other part of your program without specific reference to any specific **System** object.

System.out refers to the standard output stream, which by default is the console. **System.in** refers to the standard input which is by default the keyboard. **System.err** refers to the standard error stream, which is console by default. Of course, these streams may be redirected to any compatible I/O.

System.out and **System.err** are objects of type **PrintStream**. **System.in** is an object of type **InputStream**. Though they are used for reading and writing characters from and to the console. These are byte streams.

9.1 & 9.2 Check Your Progress.

1. Fill in the blanks.

a) ..... depend upon the Abstract Window Toolkit.

b) I/O is performed through ..... in Java.

c) At the lowest level all I/O is ..... oriented.

d) In order to use the stream classes you have to import the..... .

e) **System.out** refers to the standard output stream, which by default is the ..... .

2. Write True or False.

a) InputStream is an abstract class.

b) Reader class handles Unicode character streams.

c) System.err is object of type InputStream.

9.3 REDING CONSOLE INPUT

We saw in the preceding section that we make use of **System.in** for console input. To obtain a character based stream which is attached to the console, you wrap **System.in** in a **BufferedReader** object, to create a character stream. **BufferedReader** supports a buffered input stream. Its most commonly used constructor is :

BufferedReader(Reader *inputReader*)

where *inputReader* is the stream which is linked to the instance of BufferedReader that is being created. Remember **Reader** is an abstract class. One of its concrete classes is **InputStreamReader** which converts bytes to characters.

To get an **InputStreamReader** object which is linked to **System.in** we use the following constructor :

InputStreamReader(InputStream *inputStream*)

Now **System.in** refers to an object of type **InputStream**, so it can be used for *inputstream*. Thus we obtain the line of code to create a **BufferedReader** connected to the keyboard as follows:

BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

Now br is a character based stream linked to the console through **System.in**.

9.3.1 Reading Characters

To read a character from a **BufferedReader** we use the **read()** method as :

int read() throws IOException

Every time a **read()** is called it reads a character from the input stream and returns it as an integer value. When the end of stream is encountered it returns -1. It can throw an **IOException**.

Let us write a program to read characters from the console :

```
import java.io.*;
class CharInput {
    public static void main(String args[]) throws IOException {
        char c;
        BufferedReader br =
            new BufferedReader(new
        InputStreamReader(System.in));
        System.out.println("Enter character :");
        c = (char) br.read();
        System.out.println(c);
    }
```

```
}
```

A sample run of the program :

```
Enter character
a
a
```

Remember that **System.in** is by default line buffered. This means that no input will pass to the program till you press Enter. Thus you have to press Enter everytime you want **System.in** to read a character. **read()** is therefore not very valuable for interactive console input.

9.3.2 Reading Strings

readLine() is a method of the **BufferedReader** class which can be used to read a string from the keyboard. Its general form is :

```
String readLine() throws IOException
```

Note that it returns a **String** object. The following program demonstrates the use of **readLine()** to read and display lines of text till you enter the word Over.

```
import java.io.*;
class ReadLines {
                public static void main(String args[])
                    throws IOException
                {
                    BufferedReader br =
                        new BufferedReader(new
                    InputStreamReader(System.in));
                    String s1;
                    System.out.println("Enter Strings and Over to
                quit");
                    do {
                        s1 = br.readLine();
                        System.out.println(s1);
                    } while(!s1.equals("Over"));
                }
}
```

9.3 Check Your Progress.

1. Fill in the blanks.

a) ..... supports a buffered input stream.

b) To read a character from a BufferedReader we use the ..... method.

c) Every time a **read()** is called it reads a character from the input stream and returns it as an ..... value.

d) ..... is a method of the **BufferedReader** class which can be used to read a string from the keyboard.

9.4 WRITING CONSOLE OUTPUT

We have used **print()** and **println()** to accomplish console output in all our previous examples. These methods are defined by the class **PrintStream** which is the type of the object referenced by **System.out**. Remember that **System.out** is a byte stream. **PrintStream** also implements the low level method **write()** since it is an output stream derived from **OutputStream**. Thus, **write()** can also be used to write to the console. The simplest form of **write()** is :

void write(int *byteval*) throws IOException

This method writes to the file the byte which is specified by *byteval*. Though byteval is declared as an integer, only the lower order eight bits are written. The following example will demonstrate use of **write()** :

```
class Write {
    public static void main(String args[]) {
        int i;
        i = 'Z';
        System.out.write(i);
        System.out.write('\n');
    }
}
```

9.4.1 PrintWriter Class

The recommended method of writing to the console when using Java is through a **PrintWriter** stream. **PrintWriter** is one of the character based classes. Using a character based class for console input/output makes it easy to internationalise your programs. Therefore though **System.out** is permissible for console output, **PrintWriter** stream is what is recommended.

The constructor of **PrintWriter** which we shall use is :

PrintWriter(OutputStream *outputStream,* boolean *flushOnNewline*)

Here, *outputStream* is an object of type OutputStream and *flushOnNewline* controls whether Java flushes the output stream every time a '\n' character is output. If *flushOnNewline* is **true**, flushing automatically takes place, and if **false**, then flushing is not automatic.

PrintWriter supports the **print()** and **println()** methods for all types including **Object**. Thus these methods can be used in the same way as they have been used with **System.out**. If an argument is not a simple type, then **PrintWriter** methods call the object's **toString()** method and then print the result.

To write to the console using a **PrintWriter**, specify **System.out** for the output stream and flush the stream after each newline.

The following example demonstrates :

```
import java.io.*;
public class PrintWrite {
                    public static void main(String args[]) {
                        PrintWriter pw = new
                    PrintWriter(System.out, true);
                        pw.println("A PrintWriter Demo string");
                        int i = 10;
                        pw.println(i);
                        double d = 3.2e-4;
                        pw.println(d);
                    }
}
```

The output of the program will be :

```
A PrintWriter Demo string
10
3.2E-4
```

9.4 Check Your Progress.

1. Write True or False.

a) The **println()** method is defined by the class **PrintStream**.

b) The **write()** method cannot be used to write to the console.

c) **PrintWriter** is a character based class.

d) **PrintWriter** cannot support the **println()** method for objects.

9.5 READING AND WRITING FILES

There are a number of classes provided by Java to read and write files. In Java, all files are byte oriented and Java provides the methods to read and write bytes from and to a file. We shall study the basics of file I/O with two of the most frequently used stream classes **FileInputStream** and **FileOutputStream**. These classes create bytestreams linked to files. To open a file you create an object of one of these classes by specifying the name of the file an an argument to the constructor. The forms of these classes which we shall be using are :

FileInputStream(String *fileName*) throws FileNotFoundException

FileOuptutStream(String *fileName*) throws FileNotFoundException

fileName specifies the name of the file to be opened. The **FileNotFound Exception** will be thrown if you create an input stream and the specified file does not exist. For output streams if the file cannot be created then the **FileNotFound** Exception will be thrown. If a file with the name already exists, then its contents

will be destroyed.

You should close the file using **close()** once you have finished working with the file. This method is defined by both **FileInputStream** and **FileOutputStream** and is :

void close() throws IOException

Reading from a file :

In order to read from a file we use the following form of **read()** which is defined in the **FileInputStream**

int read() throws IOException

Every time **read()** is called it reads a single byte from the file and returns the byte as an integer value. When the end of file is encountered **read()** returns a value -1. Remember it can throw an **IOException**.

The following program uses **read()** to input and display contents of a text file :

```
import java.io.*;
class FileDemo {
    public static void main(String args[])
      throws IOException
    {
        int i;
        FileInputStream fin;
        try {
            fin = new FileInputStream(args[0]);
        } catch(FileNotFoundException e) {
        System.out.println("File Not Found");
        return;
        } catch(ArrayIndexOutOfBoundsException
    e) {
        System.out.println("No filename specified");
        return;
        }
        do {
            i = fin.read();
            if(i != -1)
                System.out.print((char) i);
        } while(i != -1);
        fin.close();
    }
```

}

Remember that in this program you will supply the filename whose contents you wish to display as a command line argument. Thus when you run this program you should type :

java FileDemo *filename*

where *filename* will be the name of the file which you wish to display. The name of the file will be passed as an argument to **FileInputStream** through args[0]. It will throw an exception if you do not give a name. Also remember that the program will throw an exception if the file is not found. So you should make use of the **try** statements as shown in order to check for these situations.

Writing to a file :

To write to a file, you make use of the **write()** method defined by the **FileOutputStream**.The simplest form of **write()** is :

void write(int byteval) throws IOException

This method writes the byte specified by byteval to the file. Although byteval is declared an integer, it will write only the low order eight bits. An **IOException** is thrown if an error occurs.

The following example copies contents of one file into another. This program should be run on the command line by specifying the name of the source file and the target file.

```
import java.io.*;
class CopyFile {
                public static void main(String args[])
                 throws IOException
                {
                        int i;
                        FileInputStream fin;
                        FileOutputStream fout;
                        try {
                                try {
                                        fin = new
                                FileInputStream(args[0]);
                                } catch(FileNotFoundException e) {
                                System.out.println("Source File not
found");
                                return;
                                }

                                try {
```

```
                    fout = new
FileOutputStream(args[1]);
                    } catch(FileNotFoundException e) {
                    System.out.println("Error opening
          target file");
                    return;
                    }
          } catch(ArrayIndexOutOfBoundsException
e) {
          System.out.println("Error ! Incorrect
usage");
          return;
          }
          try {
                    do {
                    i = fin.read();
                    if(i != -1) fout.write(i);
                    } while(i != -1);
          } catch(IOException e) {
          System.out.println("File Error");
          }
fin.close();
fout.close();
}
}
```

9.5 Check Your Progress.

1. Answer the following :

a) When will the **FileInputStream()** throw a **FileNotFoundException**?

_____

_____

b) Write the simplest form of the method **write()** when used to write to a file.

_____

_____

9.6 Summary

I/O is performed through streams in Java. A stream is an abstraction which either produces or consumes information. A stream is linked to a physical device by the Java I/O system.

Java defines two types of streams : byte streams and character streams. Byte streams provide a convenient way to handle input/output of bytes. There are useful when reading or writing binary data. Character streams provide a convenient way of handling input and output of characters. They use Unicode and can therefore be internationalised. At the lowest level all I/O is byte oriented.

9.7 CHECK YOUR PROGRESS - ANSWERS

9.1 & 9.2

1. a) applets
   b) streams
   c) byte
   d) java.io
   e) console

2. a) True
   b) True
   c) False

9.3

1. a) BufferedReader
   b) read()
   c) integer
   d) readLine()

9.4

1. a) True
   b) False
   c) True
   d) False

9.5

1. a) The **FileNotFoundException** will be thrown by the **FileInputStream()**, if you create an input stream and the specified file does not exist.

   b) The simplest form of **write()** is :void write(int byteval) throws IOException

   This method writes the byte specified by byteval to the file. Although

byteval is declared an integer, it will write only the low order eight bits. An **IOException** is thrownif an error occurs.

9.8 QUESTIONS FOR SELF - STUDY

1. **Write Short notes on :**
   a) Byte Stream Classes
   b) Character Stream Classes
   c) Predefined Streams
   d) **PrintWriter** Class
2. Describe how to obtain a character based stream linked to the console through **System.in**.
3. Explain the methods used to read characters and strings from the console using **BufferedReader**.
4. Explain the **FileInputStream()** and **FileOutputStream()** in brief.

9.9 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net

❑ ❑ ❑

Notes

Chapter : 10

Applet Programming

10.0 OBJECTIVES

Dear Students,

After Studying this chapter you will be able to

- discuss what are applets
- discuss applet architecture and the method of running applets
- discuss a set of methods which provide the basic mechanism by which the browser or applet viewer interfaces to the applet and controls its execution.
- discribe some basic applet display methods
- explain passing parameters to applets
- explain how to load data into an applet

10.1 INTRODUCTION

Applets are small applications that are accessed on an Internet server, transported over the Internet, automatically installed and run as part of a web document. Once an applet arrives on the client machine it has limited access to resources. Therefore it can produce an arbitrary multimedia user interface and run complex computations without introducing the risk of viruses or breaching data integrity.

10.2 FUNDAMENTALS

To understand applets let us begin with the following example :

```
import java.awt.*;
import java.applet.*;
public class AppletDemo extends Applet {
        public void paint(Graphics g) {
                g.drawstring("Applet Fundamentals",
50,20);
        }
}
```

From this example you can see that the applet begins with two **import** statements. The first imports the **Abstract Window Toolkit** (AWT) classes. Applets interact with the user through the AWT and not through console based I/O classes. The AWT contains support for a window based graphical interface. (AWT will be discussed in detail in later chapters). The second **import** statement is **import java.applet.***; This statement imports the applet package which contains the **Applet** class. Every applet that you create must be a subclass of **Applet**.

In the next line we declare a class called AppletDemo which extends the **Applet** class. Remember that this class must be declared as **public**, because it will be accessed by code that is outside the program.

paint() method declared in the class is a method defined by AWT and must be overridden by the applet. Each time the applet wants to redisplay its output, **paint()** should be called. Output may be required to be redisplayed for a number of reasons :

- the window in which the applet is running may be overwritten by another window and then uncovered again
- the applet window may be minimized and then restored

The **paint()** method is also called when the applet begins execution. **paint()** has one parameter **Graphics**. This contains the graphics context which describes the graphics environment in which the applet is running. This context is used whenever output to the applet is required.

drawString() is called inside **paint()**. **drawString()** is a member of the **Graphics** class. The general form of this method is :

void drawString(String *message*, int *x*, int *y*)

This method outputs a string (in this case message) beginning at the specified x, y location. Remember that the upper left hand corner is location 0,0 in Java. Thus the message Applet Fundamentals will be displayed at location 50, 20.

Note that the applet does not have a **main()** method. Instead, an applet

begins execution when the name of its class is passed to an appletviewer or to a network browser.

Compile the source code in the same way as you compile other programs. However note the two ways to run an applet :

i) Execute the applet with a Java compatible web browser like Netscape Navigator : To accomplish this you need to write a short HTML text file which contains the appropriate APPLET tag.

To execute AppletDemo the HTML file you write should be as follows :

<applet code = “AppletDemo” width = 200 height = 60>

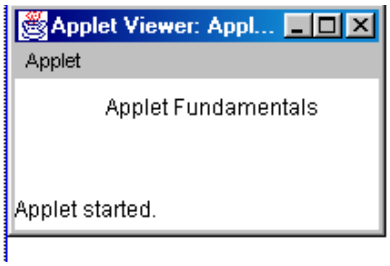
</applet>

The width and height specify the dimensions of the display area used by the applet. (The APPLET tag contains many other options which we shall discuss subsequently.) After you create this file, you can execute your browser and then load this file. It will cause AppletDemo to be executed.

ii) Use an applet viewer such as the standard JDK tool, appletviewer. An appletviewer executes the applet in a window. This is generally the fastest and easiest way to test your applet.

You can execute the above HTML file in an appletviewer also. eg. if you name your HTML file as ApDemo.html then type the following command line to run the applet :

C:\> appletviewer ApDemo.html



There is another convenient way to do this. In this method, you include a comment at the head of your Java source file that contains the APPLET tag. This documents your code with a prototype of the necessary HTML statements, and you can test your applet by starting the appletviewer with the Java sourcecode file. The revised code of your source program in this situation will be as follows :

```
import java.awt.*;
import java.applet.*;
/* <applet code = “AppletDemo” width = 200 height = 60>
</applet>
*/
public class AppletDemo extends Applet {
```

```
        public void paint(Graphics g) {
                g.drawstring("Applet Fundamentals",
50,20);
        }  }
```

Remember :

- *Applets do not need **main()** method*
- *Applets must run under an Appletviewer or a Java compatible browser*
- *User I/O is not accomplished with Java's stream I/O classes. Instead the interface provided by the AWT is used by applets for user I/O.*

10.1 & 10.2 Check Your Progress.

1. Write True or False.

a) Applets can produce an arbitrary multimedia user interface without introducing the risk of viruses .

b) Applets contain the **main()** method. .....

c) Applets interact with the user through console based I/O classes.

d) Applets can run on a Java compatible browser.

10.3 THE APPLET CLASS

The Applet class provides the necessary support for applet execution. It also provides methods that load and display images, and methods that load and display audio clips. **Applet** extends the **AWT** class **Panel** which in turn extends **Container**, which extends **Component.** These classes provide support for Java's window based graphical interface.

10.3.1 Applet Architecture

An applet is a window based program. Applets are event driven. An applet resembles a set of interrupt driven routines. An applet waits until an event has occured. The AWT notifies that applet about an event by calling an event handler that has been provided by the applet. Once, this happens the applet must take the necessary action and quickly return control to the AWT. Remember, your applet should not enter a mode of operation in which it maintains control for an extended period. Instead it must perform specific actions in response to events and return control to AWT run time system. If there are situations where your applet needs to perform repetitive tasks on its own, you must start an additional thread of execution.

The user initiates an interaction with the applet, the applet does not. These user interactions are sent to the applet as events to which the applet responds. eg. when the user clicks a mouse within an applet's window, a mouse-clicked

event is generated. Applets can contain various controls like push buttons and checkboxes. When the user interacts with these controls an event is generated.

10.3.2 An Applet Skeleton

Applets override a set of methods which provide the basic mechanism by which the browser or applet viewer interfaces to the applet and controls its execution. Four of these methods **init()**, **start()**,**stop()** and **destroy()** are defined by **Applet**. The method **paint()** is defined by the AWT **Component** class. Default implementations of these methods are provided. Also applets need not override those methods which they do not use.

Let us study these methods with the following skeleton of an applet program :

```
// An Applet Skeleton
import java.awt.*;
import java.applet.*;
/*
<applet code = “Skeleton” width = 300 height = 60>
</applet>
*/

public class Skeleton extends Applet {
    public void init() {
        // initialisation
    }
    public void start() {
        // start or resume execution
    }
    public void stop() {
        // suspends execution
    }
    public void destroy() {
        // perform shut down activity
    }
    public void paint(Graphics g) {
        // redisplay the contents of a window
    }
}
```

When an applet begins, the **AWT** calls the following methods in the given

sequence :

init() : This is the first method to be called. This is where variables should be initialised. This method is called only once during the run time of your applet.

start() : The **start()** method is called after **init()**. It is also called to restart an applet after it has been stopped. **start()** is called each time an applet's HTML document is displayed on screen. This means that if an user leaves a web page and comes back, the applet resumes execution at **start()**.

paint() : This method is called everytime the output is to be redrawn. We have already studied **paint()** in the previous section.

When an applet is terminated the following methods are called in this sequence :

stop() : This method is called when a web browser leaves the HTML document containing the applet. When **stop()** is called the applet is probably still running. **stop()** should be used to suspend threads which don't need to run when the applet is not visible. You can restart them when **start()** is called.

destroy() : This method is called when the environment determines that your applet needs to be removed completely from memory. At this point all the resources that the applet may be using should be freed. The **stop()** method is always called before **destroy()**.

update() : update() is a method defined by **AWT**. In some situations you may be required to override the **update()** method. This method is called when your applet has requested that a portion of its window be redrawn. The default version of **update()** first fills an applet with the default background colour and then calls **paint()**. If you fill the background using a different colour in **paint()** the user will experience a flash of the default background everytime **update()** is called. To avoid this, you can override the **update()** method so that it will perform the necessary activities. Then call **update()** in **paint()**. eg.

```
public void update(Graphics g) {
// redisplay your window
}
public void paint(Graphics g) {
update(g);
}
```

10.3 Check Your Progress.

1. Fill in the blanks.

a) ..... is the first method to be called in an applet program.

b) ..... method is always called before destroy().

c) ..... method is called everytime the output is to be redrawn.

2. Write True or False.

a) When **stop()** is called the applet is probably still running.

b) Applets are not event driven.

10.4 SIMPLE APPLET DISPLAY METHOD

In this section let us study some simple applet display methods.

To set the background color of an applet's window use :

setBackground(Color *newColor*)

To set the foreground color use

setForeground(Color *newColor*)

Both these methods are defined by **Component**. *newColor* specifies the new color. The class **Color** defines the constants used to specify colors.

These are :

Color.black	Color.blue	Color.cyan	Color.darkGray
Color.gray	Color.green	Color.lightGray	Color.magenta
Color.orange	Color.pink	Color.red	Color.white
Color.yellow			

You can set colors as : setForeground(Color.cyan);

setBackground(Color.green); etc

The default foreground color is black. The default background color is light gray. Foreground and background colors should generally be set in the **init()** method. They can also be changed as often as necessary during the execution of the applet.

The current settings of the background and foreground colors can be obtained by the following methods respectively :

Color getBackground();

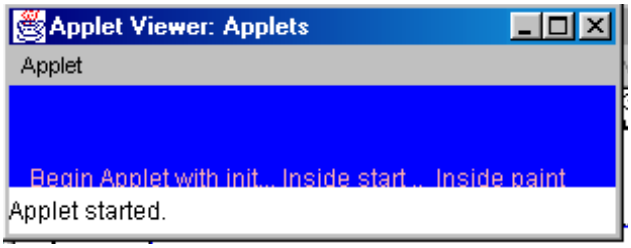
Color getForeground();

Let us study the methods to be used in applets as well as the simple methods described above with the help of the following example :

```
import java.awt.*;
import java.applet.*;
/*
<applet code = "Applets" width = 300, height = 50>
</applet>
*/
public class Applets extends Applet {
    String s1;
    public void init() {
        setBackground(Color.blue);
        setForeground(Color.pink);
```

```
                s1 = “Begin Applet with init...”;
                }
        public void start() {
                s1 += “ Inside start .. “);
                }
        public void paint(Graphics g)     {
                s1+= “ Inside paint “;
                g.drawString(s1, 10,50);
        }
}
```

stop() and **destroy()** are not needed in this applet, we have not overridden them. A sample output of the above program is :



10.5 REQUESTING REPAINTING

As a general rule, an applet writes to its window only when its **update()** or **paint()** method is called by AWT. Whenever your applet needs to update the information displayed in its window, it simply calls **repaint()**. The **repaint()** method is defined by the **AWT**. It causes the **AWT** run time system to execute a call to your applet’s **update()** method. Thus, for another part of your applet, to output to its window, simply store the output and then call **repaint()**. The **AWT** then executes a call to **paint()** which can display the stored information. The **repaint()** method has the following forms :

void repaint()

This version causes the entire window to be repainted.

void repaint(int *left*, int *top*, int *width*, int *height*)

Here, the coordinates of the upper left corner are specified by *left* and *top*, and the *width* and *height* specify the width and height of the region. Remember that these dimensions are specified in pixels. If you only need to update a small portion it is efficient to use **repaint()** only for that region with the second form of paint ().

Note that if your system is slow or busy, **update()** might not be called immediately. Such situation can cause problems especially in cases like animation where a consistent update time is necessary. For this purpose the following forms of **repaint()** can be used :

void repaint(long *maxDelay*)

void repaint(long *maxDelay*, int *x*, int *y,* int *width*, int *height*)

maxDelay specifies the maximum number of milliseconds that can elapse before **update()** is called.

10.4 & 10.5 Check Your Progress.

1. Answer the following.

a) Write the methods used to set the Background and Foreground colors in an applet.

_____

_____

b) Which is the default foreground color?

_____

_____

c) What is the use of the **repaint()** method.

_____

_____

10.6 USING THE STATUS WINDOW

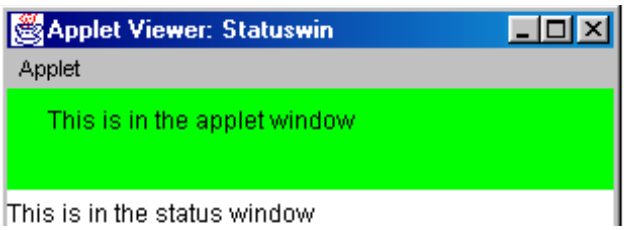
In addition to displaying information in its window, an applet can also output a message to the status window of the browser or applet viewer on which it is running. For this purpose, you make use of **showStatus()** with the string that you want to display. The status window can be used to give the user a feedback about what is occuring in the applet, suggest options or possibly report errors.

The following example demonstrates the use of **showstatus()** :

```
import java.awt.*;
import java.applet.*;
/*
<applet code = “Statuswin” width = 300 height = 50>
</applet>
*/
public class Statuswin extends Applet {
    public void init() {
        setBackground(Color.green);
    }
    public void paint(Graphics g) {
```

```
                g.drawString("This is in the applet window ",
20, 20);
                showStatus("This is in the status window");
            }
        }
```

output



10.7 PASSING PARAMETERS TO APPLETS

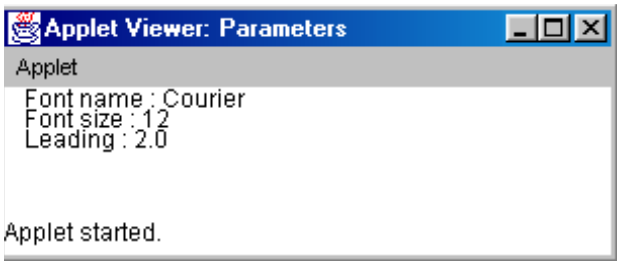
The **APPLET** tag in HTML allows you to pass parameters to your applet. To retrieve a parameter, you have the **getParameter()** method, which returns the value of the specified parameter in the form of a **String** object. Therefore numeric and **Boolean** values have to be converted from their string representations to their internal formats. The following example will illustrate :

```
import java.awt.*;
import java.applet.*;
/*
<applet code = "Parameters" width = 300 height = 60>
<param name = fontName value = Courier>
<param name = fontSize value = 12>
<param name = leading value = 2>
</applet>
*/
public class Parameters extends Applet {
String fontName;
int fontSize;
float leading;
public void start() {
        String p;
        fontName = getParameter("fontName");
        if(fontName == null)
                fontName = "Not found";
        p = getParameter("fontSize");
```

```
        try {
                if(p != null)
                        fontSize = Integer.parseInt(p);
                else
                        fontSize = 0;
                }
                catch(NumberFormatException e) {
                        fontSize = -1;
                }
                p = getParameter("leading");
                try {
                if(p != null)
                        leading = Float.valueOf(p).floatValue();
                else
                        leading = 0;
                }
                catch(NumberFormatException e) {
                        leading = -1;
                }
        }
        public void paint(Graphic g) {
                g.drawString("Font name : " + fontName, 10, 10);
                g.drawString("Font size : " + fontSize, 10, 20);
                g.drawString("Leading : "+ leading 10, 30);
        }
}
```

output

It is important to note that in this program we have checked the return values from **getParameter()**. If a parameter is not available, **getParameter()** returns a **null**. Also note that the conversion from **String** type to numeric is done in a **try** statement which catches the **NumberFormatException**. There should be no uncaught exceptions in an applet.



10.8 LOADING DATA INTO APPLET

getDocumentBase() and **getCodeBase()** functions are used to explicitly load media and text from applets. Java allows the applet to load data from the directory holding the HTML file which started the applet (the document base) and the directory from which the applet's class file was loaded (the codebase). These directories are returned as URL objects by **getDocumentBase()** and **getCodeBase()** respectively. They can be concatenated with a string which names the file you want to load. To actually load another file, you use the **showDocument()** method which is defined by the **AppletContext** interface. **AppletContext** is an interface which lets you get information from the applet's execution environment. Once you have obtained the applet's context, you can bring another document into view by calling **showDocument()**. This method has no return values and it does not throw any exception if it fails.

There are two **showDocument()** methods :

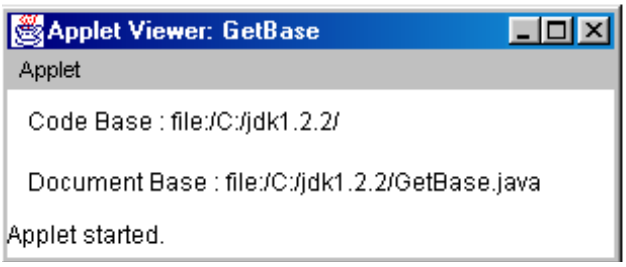
showDocument(URL) : displays the document specified by URL.

showDocument(URL, *where*) : displays the document at the location specified within the browser window by *where*. The valid arguments for where are : _self (current frame), _parent (parent frame), _top (topmost frame), _blank (new browser window).

The following program shows how to use the **getDocumentBase()** and **getCodeBase()** :

```
import java.awt.*;
import java.applet.*;
import java.net.*;
/*
<applet code = "GetBase" width = 300 height = 60 >
</applet>
*/
public class GetBase extends Applet {
        public void paint(Graphics g) {
                String s1;
                URL url = getCodeBase();
                s1 = "Code Base : " + url.toString();
                g.drawString(s1, 10, 20);
                url = getDocumentBase();
                s1 = "Document Base : " + url.toString();
                g.drawString(s1, 10, 40);
        }
}
```

The output of the program will be :



The **AudioClipInterface** defines the methods : **play()** - play a clip from the beginning, **stop()** - stop playing the clip, and **loop()** - play the loop continuously. These methods can be used to play audio clips once you have loaded an audio clip with the **getAudioClip()** method.

The **AppletStub** interface provides the means by which an applet and the browser or appletviewer communicate. This interface is normally not implemented in your code.

10.6 to 10.8 Check Your Progress.

1. Fill in the blanks.

a) The ..... window can be used to give the user a feedback about what is occuring in the applet.

b) The ..... tag in HTML allows you to pass parameters to your applet.

c) ..... and ..... functions are used to explicitly load media and text from applets.

10.9 SUMMARY

Applets are small applications that are accessed on an Internet server, transported over the Internet, automatically installed and run as part of a web document. Once an applet arrives on the client machine it has limited access to resources. Therefore it can produce an arbitrary multimedia user interface and run complex computations without introducing the risk of viruses or breaching data integrity.

Applets override a set of methods which provides the basic mechanism by which the browser or applet viewer interfaces to the applet and controls its execution. Some of these methods init(), start(),stop(), paint() and destroy() are defined by Applet.

Source : *www.scribd.com(Link)*

10.10 CHECK YOUR PROGRESS - ANSWERS

10.1 & 10.2

1. a) True b) False c) False

d) True

10.3

1. a) init()

b) stop()

c) paint()

2. a) True

b) False

10.4 & 10.5

1. a) setBackground(Color *newColor*) and setForeground(Color *newColor*) are the methods to set the Background and Foreground colors in an applet.

b) The default foreground color is black.

c) Whenever an applet needs to update the information displayed in its window it makes use of the **repaint()** method.

10.6 to 10.8

1. a) status

b) APPLET

c) getDocumentBase(), getCodeBase()

10.11 QUESTIONS FOR SELF - STUDY

1. What is the **paint**() method? Give reasons why an output may be required to be repainted?
2. Describe the ways to run an applet.
3. Describe the skeleton of an applet program.
4. Describe some simple applet display methods.
5. **Write short notes on :**

   a) Applet Architecture

   b) The repaint() method

   c) Loading Data into an Applet

6. What is the use of Status window? Describe with example the **showStatus**() method.

10.12 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net

❑ ❑ ❑

Notes

Notes

Chapter : 11

Event Handling

11.0 OBJECTIVES

Dear Students,

After studying this chapter you will able to :

- Explain the meaning of what is an event
- Explain the Delegation Event Model
- discuss some important Event Classes
- discuss some sources of events and the event listener interfaces
- discuss some sample programs for handling simple events.

11.1 INTRODUCTION

An important aspect of Java that relates to applets is **events**. Applets are event driven programs. The most commonly handled events are those generated by the mouse, the keyboard, and the various controls like push buttons. Events are supported by the **jawa.awt.event** package.

11.2 THE DELEGATION EVENT MODEL

The delegation event model defines standard and consistent mechanisms to generate and process events. In this model, the concept is, that a source generates an event and sends it to one or more listeners. The listener simply waits for an event, once it receives the event, the listener processes it and returns. The advantage of this design is that the application logic which processes the events is cleanly separated from the user interface logic, that generates the events. Thus the user interface element is able to delegate the processing of an event to a separate piece of code. In this model, the listeners must register with a source in order to receive the notification of an event. This means that notifications are sent only to those listeners who wish to receive them.

11.2.1 Events

In the delegation model, an event is an object which describes a state change in a source. The event can be generated as a result of a person interacting with the elements in a graphical user interface. Pressing a button, entering a character via a keyboard, selecting an item in a list, clicking the mouse are some of the activities which cause events to be generated. Events may also occur as a consequence which is not directly caused by user interface. eg. an event may be generated if a timer expires, a counter exceeds a value, a hardware or software failure occurs etc. You are free to define events that are appropriate to your application.

11.2.2 Sources of Events

A source is an object which generates an event. This occurs when the internal state of the object changes in some way. Sources may generate more than one type of event. A source must register listeners in order for the listeners to receive notifications about a specific type of event. Each type of event has its own registration method.

The general form is :

public void add*Type*Listener(*Type*Listener *el*)

where *Type* is the name of the event and *el* is the reference to the event listener. eg the method that registers a keyboard event listener is called **addKeyListener()**.

When an event occurs, all the registered listeners are notified and receive a

copy of the event object. This is known as ***multicasting*** the event.

Some sources may allow only one listener to register.

The general form of this method is :

public void addTypeListener(*Type*Listener *el*)

throws java.util.TooManyListenersException

where *Type* is the name of the event and *el* is the reference to the event listener. When such an event occurs, the registered listener is notified. This is called as **unicasting** the event.

A source must also provide a method to allow a listener to unregister an interest in a specific event. The general form of this method is :

public void remove *Type*Listener(*Type*Listener *el*)

here, *Type* is the name of the event and *el* is the reference to the event listener eg to remove a keyboard listener you would call the **removeKeyListener()** method.

The methods that add or remove listeners are provided by the source that generates the events.

11.2.3 Event Listeners

A listener is an object that is notified when an event occurs. It has two major requirements :

- First it must have been registered with one or more sources to receive the notifications about specific types of event.
- Second it must implement methods to receive and process these notifications.

The methods which receive and process events are defined in a set of interfaces found in **java.awt.event**.

11.1 & 11.2 Check Your Progress.

1. Fill in the blanks.

a) Events are supported by the ..... package.

b) In the delegation model, an ..... is an object which describes a state change in a source.

c) A ..... is an object which generates an event.

d) A ..... is an object that is notified when an event occurs.

2. Write True or False.

a) Pressing a button causes events to be generated.

b) A listener need not have been registered with one or more sources to receive the notifications about specific types of event.

11.3 EVENT CLASSES

11.3.1 Introduction

The classes which represent events are at the core of Java's event handling mechanism. At the root of the Java class event hierarchy is the **EventObject**, which is in **java.util**. It is the superclass for all events. One of its constructors is :

EventObject(Object *src*)

where *src* is the object that generates the event.

EventObject contains two methods : **getSource()** and **toString()**. **getSource()** method returns the source of the event. Its general form is :

Object getSource()

toString() returns the string equivalent of the event.

The class **AWTEvent**, which is defined in the **java.awt package**, is a subclass of **EventObject**. It is the superclass of all AWT-based events used by the delegation event model. The method **getID()** of this class is used to determine the type of the event.

Its form is :

int getID()

The package **java.awt.event** defines several types of events that are generated by various user interface elements. Some of the most important of these event classes are given in the table below :

Event Class	Description
Action Event	Generated when a button is pressed, a list item is double clicked or a menu item is selected
Adjustment Event	Generated when a scrollbar is manipulated Component Event Generated when a component is hidden, moved, resized or becomes visible
Container Event	Generated when a component is added to or removed from a container Focus Event Generated when a component gains or loses keyboard focus

Input Event	Abstract superclass for all component input event classes
	Item Event Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected
Key Event	Generated when input is received from the keyboard
Mouse Event	Generated when the mouse is dragged, moved, clicked, pressed, or released, also generated when the mouse enters or exits a component
Text Event	Generated when the value of a text area or text field is changed
Window Event	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened or quit

11.3.1 Check Your Progress.

1. Answer the following.

a) List the methods of the **EventObject** class.

_____

_____

b) Describe the following event classes :

(i) ActionEvent : _____

_____

ii) Mouse Event : _____

_____

Let us briefly go through some of the classes listed in the previous section and some methods of these classes :

11.3.2 The Action Event Class

From the table we can see that an **ActionEvent** is generated when a button

is pressed, a list item is double clicked or a menu item is selected. This class defines four integer constants which can be used to identify any modifiers associated with an action event. These constants are **ALT_MASK**, **CTRL_MASK**, **META_MASK**, **SHIFT_MASK**. There is also an integer constant **ACTION_PERFORMED**, which can be used to identify action events.

The constructors of **ActionEvent** are :

ActionEvent(Object *src*, int *type*, String *cmd*)

ActionEvent(Object *src*, int *type*, String *cmd*, int *modifiers*)

where *src* is a reference to the object which generated the event. *type* specifies the type of the event and *cmd* is the command string. The argument *modifiers* indicates which of the modifier keys (ALT, CTRL, META and/or SHIFT) were pressed when the event was generated.

getActionCommand() is the method which used to obtain the command name for the invoking **ActionEvent** object. Its form is :

String getActionCommand()

The **getModifiers()** method returns a value that indicates which modifier keys were pressed when the event was generated. Its general form is :

int getModifiers()

11.3.3 The Component Event Class

This class is generated when the size, position or visibility of a component is changed. There are four types of component events. **The ComponentEvent** class defines integer constants that can be used to identify these component events. The constants and their meanings are :

COMPONENT_HIDDEN	The component was hidden
COMPONENT_MOVED	The component was moved
COMPONENT_RESIZED	The component was resized
COMPONENT_SHOWN	The component became visible

The constructor for the class is :

ComponentEvent(Component *src*, int *type*)

where *src* is a reference to the object that generated this event and *type* specifies the type of the event.

ComponentEvent is the superclass directly or indirectly of **ContainerEvent**, **FocusEvent**, **KeyEvent**, **MouseEvent** and **WindowEvent**. Its method **getComponent()** returns the component which generated the event and its general form is :

Component getComponent()

11.3.4 The Container Event Class :

A **ContainerEvent** is generated when a component is added to or removed from a container. There are two types of container events. The class

defines integer constants that can be used to identify them : **COMPONENT_ADDED** and **COMPONENT_REMOVED**. They indicate that a component has been added or removed from the container.

ContainerEvent is a subclass of **ComponentEvent**. Its constructor is :

ContainerEvent(Component *src*, int *type*, Component *comp*)

where *src* is a reference to the container that generated this event, the type of the event is specified by *type*, and the component that has been added or removed is specified by *comp*.

With the **getContainer()** method you can obtain a reference to the container that generated this event. The method has the general form :

Container getContainer()

The **getChild()** is the method which returns a reference to the component that was added to or removed from the container. Its general form is :

Component getChild()

11.3.2 to 11.3.4 Check Your Progress

1. Fill in the blanks.

a) ..... is the method which used to obtain the command name for the invoking **ActionEvent** object.

b) The ..... class is generated when the size, position or visibility of a component is changed.

c) The two constants used to identify container events are ..... and ..... .

11.3.5 The Input Event Class

This abstract class is a subclass of **ComponentEvent** and is the superclass for component input events. **KeyEvent** and **MouseEvent** are its subclasses. The **InputEvent** class defines the following eight integer constants that can be used to obtain information about any modifiers associated with this event :

ALT_MASK	**BUTTON2_MASK**	**META_MASK**
ALT_GRAPH_MASK	**BUTTON3_MASK**	**SHIFT_MASK**
BUTTON1_MASK	**CTRL_MASK**	

The methods **isAltDown()**, **isAltGraphDown()**, **isControlDown()**, **isMetaDown()** and **isShiftDown()** are used to test whether these modifiers were pressed at the time this event was generated. All these methods return a boolean value.

The **getModifiers()** method returns a value that contains all of the modifier flags for this event. Its form is :

int getModifiers()

11.3.6 The Item Event Class

An ItemEvent is generated when a check box or list item is clicked or when

a checkable menu item is selected or deselected. There are two types of item events and they are identified by the following integer constants :

DESELECTED	The user deselects an item
SELECTED	The user selects an item

This class also defines one integer constant **ITEM_STATE_CHANGED** which signifies a change of state. **ItemEvent** has the following constructor :

ItemEvent(ItemSelectable *src*, int *type*, Object *entry*, int *state*)

src is a reference to the component that generated the event, *type* specifies the type of the event. The specific item that generated the item event is passed in *entry*, and the current state of that item is passed in *state*.

In order to obtain a reference to the item that generated an event we use the method **getItem()**. Its general form is :

Object getItem()

The **getStateChanged()** method returns the state changed for the event. Its form is :

int getStateChanged()

where it returns **SELECTED** or **DESELECTED**.

The **getItemSelectable()** method is used to obtain a reference to the **ItemSelectable** object that generated the event. Its form is :

ItemSelectable getItemSelectable()

11.3.7 The Key Event Class

A **KeyEvent** is generated when a keyboard input occurs. There are three types of key events which are identified by the integer constants : **KEY_PRESSED**, **KEY_RELEASED**, **KEY_TYPED**. The first two events are generated when any key is pressed or released. The last event occurs only when a character is generated. We know that all key presses do not result in characters eg. pressing the SHIFT key will not generate any character.

KeyEvent defines many other integer constants. eg. **VK_0** through **VK_9** and **VK_A** through **VK_Z** define the ASCII equivalents of the numbers and literals respectively. Some other constant are :

where the **VK** constants specify the virtual key codes and are independant of any modifiers like control, shift or alt.

KeyEvent is a subclass of **InputEvent**. Its constructors are :

KeyEvent(Component *src*, int *type*, long *when*, int *modifiers*, int *code*)

KeyEvent(Component *src*, int *type*, long *when*, int *modifiers*, int *code*, char *ch*)

where *src* is a reference to the component that generated the event, type of the event is specified by *type*. The system time at which the key was pressed is passed in *when*. The *modifiers* argument indicates which modifiers were pressed when this key event occured. The virtual key code is passed in *code*. The

character equivalent, if it exists is passed in *ch*. If no valid character exists, then *ch* contains **CHAR_UNDEFINED**.

The commonly used methods of the **KeyEvent** class are :

getKeyChar() : which returns the character that was entered and has the following general form :

char getKeyChar()

getKeyCode() : which returns the key code. Its general form is :

int getKeyCode()

11.3.5 to 11.3.7 Check Your Progress.

1. Fill in the blanks.

a) ..... and ..... are the subclasses of the InputEvent class.

b) The two types of item events are identified by the integer constants ..... and ..... .

c) A..... is generated when a keyboard input occurs.

d) There are three types of key events which are identified by the integer constants : ....., ... and ..... .

11.3.8 The Mouse Event Class :

There are seven types of mouse events. The following integer constants are used to identify them :

MOUSE_CLICKED **MOUSE_DRAGGED** **MOUSE_ENTERED**
MOUSE_EXITED **MOUSE_MOVED** **MOUSE_PRESSED**
MOUSE_RELEASED

MouseEvent is a subclass of **InputEvent** and has the following constructor:

MouseEvent(Component *src*, int *type*, long *when*, int *modifiers*,
int *x*, int *y*, int *clicks*, boolean *triggersPopup*)

Here *src* is a reference to the component that generated the event. The type of the event is specified by *type*. *when* specifies the system time at which the mouse event occured. The *modifiers* argument indicates which modifiers were pressed when a mouse event occured. The coordinates of the mouse are passed in *x* and *y*. *clicks* specifies the click count. The *triggersPopup* flag indicates if this event causes a pop up menu to appear on this platform.

The most commonly used method of this class are **getX()** and **getY()** which return the coordinates of the mouse when the event occured. Their general forms are :

int getX()

int getY()

The **getPoint()** method returns the coordinates of the mouse. This method has the following form:

Point getPoint()

It returns a **Point** object, which contains the X, Y coordinates in its integer members x and y.

translatePoint() method changes the location of the event. Its form is :

void translatePoint(int *x*, int *y*)

The arguments *x* and *y* are added to the coordinates of the event.

11.3.9 The Window Event Class

There are seven types of window events and the **WindowEvent** class defines integer constants to identify them. The constants are :

WINDOW_ACTIVATED **WINDOW_CLOSED**

WINDOW_CLOSING **WINDOW_DEACTIVATED**

WINDOW_DEICONIFIED **WINDOW_ICONIFIED**

WINDOW_OPENED

WindowEvent is a subclass of **ComponentEvent** and its constructor is :

WindowEvent(Window *src*, int *type*)

where *src* is a reference to the object that generated this event and *type* is the type of the event.

getWindow() is the most commonly used method of this class and it returns the **Window** object that generated this event. Its general form is :

Window getWindow()

11.3.8 & 11.3.9 Check Your Progress.

1. Fill in the blanks.

a) There are ..... types of mouse events.

b) ..... is the most commonly used method of the WindowEvent class.

11.4 SOURCES OF EVENTS

The table given on the next page lists some of the user interface components that can generate the events described in the previous section. In addition to these, other components like applets can also generate events. You can also build your own components that generate events.

Event Source	Description
Button is	Generates action events when the button pressed
Checkbox box	Generates item events when the check is selected or deselected
Choice	Generates item events when the choice is changed
List	Generates action events when an item is double clicked, generates an item event when an item is selected or deselected
Menu Item	Generates action events when a menu item is selected, generates item events when a checkable menu item is selected or deselected
Scrollbar	Generates adjustment events when the scroll bar is manipulated
Text components	Generates text events when the user enters a character
Window	Generates window events when a window is activated, closed, deactivated, diconified, iconified, opened or quit

11.5 EVENT LISTENER INTERFACES

Listeners are created by implementing one or more of the interfaces defined by the **java.awt.event** package. When an event occurs, the event source invokes the appropriate method defined by the listener and provides an event object as its argument. Let us study some of the listener interfaces and the specific methods contained in them in this section :

The ActionListener Interface :

This interface defines one method to receive action events. The method is **actionPerformed()** and is invoked when an action event occurs. Its general form is :

void actionPerformed(ActionEvent *ae*)

The ComponentListener Interface :

This interface defines four methods that are invoked when a component is resized, moved, shown or hidden. The general forms of these methods are :

void ComponentResized(ComponentEvent *ce*)

void ComponentMoved(ComponentEvent *ce*)

void ComponentShown(ComponentEvent *ce*)

void ComponentHidden(ComponentEvent *ce*)

The ContainerListener Interface :

This interface contains two methods **componentAdded()** and **componentRemoved()**. **componentAdded()** is invoked when a component is added to the container, and **componentRemoved()** is invoked when the component is removed from the container. The general forms of these methods are :

void ComponentAdded(ContainerEvent *ce*)

void ComponentRemoved(ContainerEvent *ce*)

ItemListener Interface :

This defines the i**temStateChanged()** method that is invoked when the state of an item changes. Its general form is :

void itemStateChanged(ItemEvent *ie*)

The KeyListenerInterface : This interface defines three methods

- **keyPressed()** and **keyReleased()** are invoked when a key is pressed and released respectively. The general forms of these methods are :

void KeyPressed(KeyEvent *ke*) void KeyReleased(KeyEvent *ke*)

- **keyTyped()** method is invoked when a character has been entered.

Its general form is :

void keyTyped(KeyEvent *ke*)

The MouseListenerInterface :

This interface defines five methods. If the mouse is pressed and released at the same point, **mouseClicked()** is invoked. Its general form is :

void mouseClicked(MouseEvent *me*)

When the mouse enters a component the **mouseEntered()** method is called. Its form is :

void mouseEntered(MouseEvent *me*)

When it leaves a component the **mouseExited()** is called. Its general form is :

void mouseExited(MouseEvent *me*)

The **mousePressed()** and **mouseReleased()** methods are invoked when the mouse is pressed and released respectively. Their general forms are :

void mousePressed(MouseEvent *me*)

void mouseReleased(MouseEvent *me*)

The MouseMotionListener Interface :

This interface defines two methods. The **mouseDragged()** is called multiple times as the mouse is dragged. The **mouseMoved()** method is called multiple times as the mouse is moved. Their general forms are :

void mouseDragged(MouseEvent *me*)

void mouseMoved(MouseEvent *me*)

The WindowListener Interface :

The interface defines seven methods. The **windowActivated()** and **windowDeactivated()** methods are invoked when a window is activated or deactivated respectively. If a window is iconified, the **windowIconified()** method is called and when it is deiconified, the **windowDeiconified()** method is called. When a window is opened or closed, the **windowOpened()** or **windowClosed()** methods are called, respectively. The **windowClosing()** method is called when a window is being closed. The general forms of these methods are:

void windowActivated(WindowEvent *we*)

void windowClosed(WindowEvent *we*)

void windowClosing(WindowEvent *we*)

void windowDeactivated(WindowEvent *we*)

void windowDeiconified(WindowEvent *we*)

void windowIconified(WindowEvent *we*)

void windowOpened(WindowEvent *we*)

11.5 Check Your Progress.

1. Match the following.

Column A	**Column B**
a) actionPerformed()	(i) MouseMotionListener interface
b) componentAdded()	(ii) ItemListener interface
c) mouseDragged()	(iii) ContainerListener interface
d) mouseClicked()	(iv) ActionListener interface
e) itemStateChanged()	(v) MouseListener interface

11.6 HANDLING EVENTS

In this section, we shall study examples which will handle the two most commonly used event generators : the keyboard and the mouse. We shall use the delegation event model in applet programming examples. The steps are :

- Implement the appropriate interface in the listener so that it will receive the type of the event desired.

- Implement the code to register and unregister (if required) the listener as a recipient for the event notifications.

Remember that a source may generate several types of events. Each event must be registered separately. Also, an object may register to receive several types of events and therefore it must implement all of the interfaces that are required to receive these events.

11.6.1 Handling Keyboard events

The following example will demonstrate how to handle simple keyboard

events. Let us briefly review how the keyboard events are generated. When a key is pressed a **KEY_PRESSED** event is generated and the **keyPressed()** event handler is called. Similarly when a key is released, a **KEY_RELEASED** event is generated and the **keyReleased()** handler is executed. If a keystroke generates a character then the **KEY_TYPED** event is generated and the **keyTyped()** handler invoked. Thus, with every key press at least two or many times three events are generated. If your program needs to handle special keys like arrow or function keys then the **keyPressed()** handler should be used. An important point to remember is that in order for your program to receive keyboard events, it must request input focus. For this purpose, call the **requestFocus()** which is defined by **Component**. Else, no keyboard events will be received.

```
// Demonstrate keyboard events
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = “KeyEvents” width = 300 height = 60>
</applet>
*/
public class KeyEvents extends Applet implements
KeyListener {
        String msg = “ “;
        int X = 10, Y = 40;     // set coordinates to output
        public void init() {
                addKeyListener(this);
                requestFocus();    // request input focus
        }
                public void keyPressed(KeyEvent ke) {
                showStatus(“Key Pressed”);
        }

        public void keyReleased(KeyEvent ke) {
                showStatus(“Key Released”);
        }

        public void keyTyped(KeyEvent ke) {
                msg+= ke.getKeyChar();
                repaint();
```

```
        }

        public void paint(Graphics g) {
            g.drawString(msg, X, Y);                    //
display the key typed
        }
    }
```

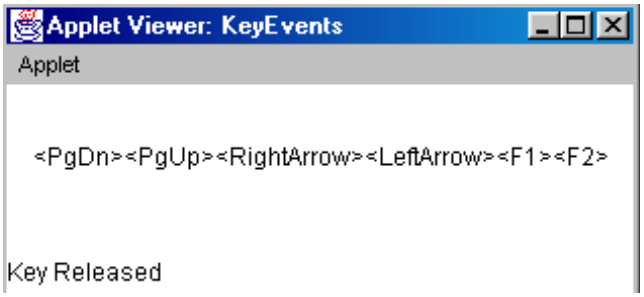
Run this program and check the status when the keyboard keys are pressed and released.

In order to handle special keys like the function keys or the arrow keys you are required to use the **keyPressed()** handler. The **keyTyped()** handler does not provide them. Also to identify these you have to use their virtual key codes. Modify the **keyPressed()** handler of your above program as shown below to illustrate how to respond to special keys.

```
public void keyPressed(KeyEvent ke) {
        showStatus("KeyPressed");
        int k = ke.getKeyCode();
        switch(k) {
            case KeyEvent.VK_PAGE_UP:
                msg += "<PgUp>";
                break;
            case KeyEvent.VK_PAGE_DOWN:
                msg += "<PgDn>";
                break;
            case KeyEvent.VK_LEFT:
                msg += "<LeftArrow>";
                break;
            case KeyEvent.VK_RIGHT:
                msg += "<RightArrow>";
                break;
            case KeyEvent.VK_F1 :
                msg += "<F1>";
                break;
            case KeyEvent.VK_F2 :
                msg += "<F2>";
                break;
            }
        repaint();
```

}

Sample output of the above program is as shown below :



11.6.2 Handling Mouse Events

For handling mouse events you have to implement the **MouseListener** and the **MouseMotionListener** interfaces. The following program shows the current coordinates of the mouse in the status window of the applet. Everytime a button is pressed the word “Down” is displayed at the current mouse pointer location and everytime a button is released, the word “Up’ is shown. If a button is clicked the message “Mouse Clicked” is displayed in the upper left corner of the applet display area. Also whenever a mouse enters or exits the applet window a message is displayed in the upper left hand corner. When a mouse is dragged a * is displayed which tracks the mouse pointer as it is dragged.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = “MouseEvents” width = 300 height = 60>
</applet>
*/
public class MouseEvents extends Applets implements MouseListener,
MouseMotionListener {
        String msg = “ “;
        int X = 0, Y = 0;           // Mouse coordinates
        public void init() {
                addMouseListener(this);
                addMouseMotionListener(this);
        }
        public void mouseClicked(MouseEvent me) {
```

```
        X = 0;
        Y = 10;
        msg = “Mouse Clicked”;
        repaint();
    }
    public void mouseEntered(MouseEvent me) {
        X = 0;
        Y = 10;
        msg = “Mouse entered”;
        repaint();
    }
    public void mouseExited(MouseEvent me) {
        X = 0;
        Y = 10;
        msg = “Mouse Exited”;
        repaint();
    }
    public void mousePressed(MouseEvent me) {
        X = me.getX();
        Y = me.getY();          // get coordinates
        msg = “Down”;
        repaint();
    }
    public void mouseReleased(MouseEvent me) {
        X = me.getX();
        Y = me.getY();          // get coordinates
        msg = “Up”;
        repaint();
    }
    public void mouseDragged(MouseEvent me) {
        X = me.getX();
        Y = me.getY();          // get coordinates
        msg = “*”;
        showStatus(“Dragging Mouse” + X+ “,”+ Y);
        repaint();
    }
    public void mouseMoved(MouseEvent me) {
```

```
            showStatus("Moving Mouse");
        }
        public void paint(Graphics g) {
            g.drawString(msg, X, Y);
        }
    }
```

run above program and study output

Note that the **MouseEvents** class extends **Applet** and implements both **MouseListener** and **MouseMotionListener** interfaces. These interfaces contain methods which receive and process the mouse events. Each method handles its event and then returns.The applet is both the source and the listener of the events, because **Component** which supplies the **addMouseListener()** and **addMouseMotionListener()** methods is a superclass of **Applet**.

11.7 ADAPTER CLASSES

Adapter classes is a special feature of Java. It helps to simplify event handlers in certain situations. An adapter class is a class which provides an empty implementation of all the methods in an event listener interface. Thus these classes are useful in situations when you want to receive and process only some of the events that are handled by a particular event listener interface. You can define a new class to act as an event listener by extending one of the adapter classes and implementing only those events in which you are interested. For example, the **Mouse Motion Adapter** class has two methods **mouse Dragged()** and **mouse Moved()** whose signatures are exactly as defined in the **MouseMotionListener** interface. Suppose you are interested only in mouse dragged events, then you can simply extend **Mouse Motion Adapter** and implement **mouse Dragged ()**. The empty implementation of **mouse Moved ()** would handle the mouse events.

The following table shows the different adapter classes in java.awt.event and the interface that each implements.

Adapter Class	Listener Interface
Component Adapter	Component Listener
Container Adapter	Container Listener
Focus Adapter	Focus Listener
Key Adapter	Key Listener
Mouse Adapter	Mouse Listener
Mouse Motion Adapter	Mouse Motion Listener
Window Adapter	Window Listener

11.6 & 11.7 Check Your Progress.

1. Fill in the blanks.

a) In order for your program to receive keyboard events, it must request input focus by calling the ..... method.

b) In order to handle special keys like the function keys or the arrow keys you are required to use the ..... handler.

c) An ..... class provides empty implementations of all methods in an event listener interface.

d) The ..... adapter class implements the Window Listener interface.

11.8 SUMMARY

Applets are event driven programs. The most commonly handled events are those generated by the mouse, the keyboard, the various controls like push buttons. Events are supported by the jawa.awt.event package.

The delegation event model defines standard and consistent mechanisms to generate and process events. In the delegation model, an event is an object which describes a state change in a source.

11.9 CHECK YOUR PROGRESS - ANSWERS

11.1 & 11.2

1. a) java.awt.event b) event

c) source d) listener

2. a) True b) False

11.3.1

1. a) **EventObject** contains two methods : **getSource()** and **toString()**.

b) i) ActionEvent : Generated when a button is pressed, a list item is double clicked or a menu item is selected

ii) MouseEvent :Generated when the mouse is dragged, moved, clicked, pressed, or released, also generated when the mouse enters or exits a component

11.3.2 to 11.3.4

1. a) getActionCommand()

b) ComponentEvent

c) COMPONENT_ADDED, COMPONENT_REMOVED

11.3.5 to 11.3.7

1. a) KeyEvent, MouseEvent

b) DESELECTED, SELECTED

c) KeyEvent

d) KEY_PRESSED, KEY_RELEASED, KEY_TYPED

11.3.8 & 11.3.9

1. a) seven

b) getWindow()

11.4

1. a) Generates action events when the button is pressed

b) Generates adjustment events when the scroll bar is manipulated

11.5

1. a) - (iv)

b) - (iii)

c) - (i)

d) - (v)

e) - (ii)

11.6 & 11.7

1. a) requestFocus() c) Adapter

b) keyPressed() d) Window Adapter

11.10 QUESTIONS FOR SELF - STUDY

1. Describe the Delegation Event Model in detail.
2. What do you understand by multicasting and unicasting?
3. Describe in detail any five Event classes and some of the methods of these classes.
4. Describe the various user interface components that can generate events.
5. List the steps to handle event generators using the delegation event model.
6. Describe any four EventListener interfaces and the specific methods contained in them.

11.11 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net

❑ ❑ ❑

Notes

Notes